

Comprehensive Study Guide

Covers:

- Relational Model & Relational Algebra
- Database Design & Normal Forms
- SQL: Queries, DDL, DML, Views, Triggers & Assertions

Based on course slides & lecture materials

1. Introduction to the Relational Model

The relational model was first introduced by Ted Codd of IBM in 1970. It uses the concept of a mathematical relation (table of values) as its basic building block. Its theoretical basis comes from set theory and first-order predicate logic.

Major commercial DBMSs implementing this model include Oracle, DB2, SQL Server, Informix, and Microsoft Access.

1.1 Core Terminology

Relation (Table)

A set of tuples (rows) sharing the same attributes. Represents a real-world entity or relationship.

Tuple (Row)

A single record in a relation — an ordered list of n values corresponding to the n attributes.

Attribute (Column)

A named property of a relation. Each attribute has a domain — the set of valid values it can take.

Domain

A set of atomic (indivisible) values. Defined by a name, data type, and format. E.g., Phone_numbers: 10-digit character strings.

Degree (Arity)

The number of attributes in a relation schema. A STUDENT relation with 7 attributes has degree 7.

1.2 Relation Schema & State

A relation schema R , denoted $R(A_1, A_2, \dots, A_n)$, is the structure — the schema name plus attribute names. For example:

`STUDENT (Name, SSN, HomePhone, Address, OfficePhone, Age, GPA)`

A relation state $r(R)$ is the actual set of tuples at a point in time. A tuple is written as:

`t = <v1, v2, ..., vn>    where vi is from dom(Ai) or is null`

To reference a specific value: $t[A_i]$ or $t.A_i$. Multiple attributes: $t[A_u, A_w, A_z]$.

1.3 Characteristics of Relations

Ordering of Tuples

A relation is a set of tuples — there is no inherent ordering. Two states listing tuples in different sequences are considered identical.

Ordering of Values within a Tuple

An n-tuple is an ordered list of n values. The order of attribute-value pairs matters, but at a logical level, correspondence between attribute name and value is what counts.

Values and Nulls

Each value in a tuple must be atomic (indivisible) — this is why the model is called the flat relational model. Composite and multivalued attributes are not permitted.

NULL is a special marker used when: (a) a value is unknown, (b) a value is not applicable, or (c) it is unavailable.

Interpretation of a Relation

A relation schema can be read as a declaration or predicate. Each tuple is a fact satisfying that predicate. For example, MAJORS(StudentSSN, DeptCode) asserts which students major in which departments.

1.4 Relational Model Notation

- Q, R, S — relation names
- q, r, s — relation states
- t, u, v — tuples
- t[Ai] or t.Ai — value of attribute Ai in tuple t
- R.A — qualified attribute name using dot notation (e.g., STUDENT.Name)

2. Relational Model Constraints

Constraints restrict valid values in a database. They fall into three categories:

- Inherent model-based constraints — built into the model itself
- Schema-based constraints — expressible in the schema (domain, key, integrity)
- Application-based constraints — enforced by programs (triggers, assertions)

2.1 Domain Constraints

Each attribute value in a tuple must be an atomic value from its defined domain $\text{dom}(A)$. Data types include integers, floats, fixed/variable character strings, dates, times, and enumerated types.

2.2 Key Constraints

Superkey

A superkey SK ensures no two distinct tuples in any state of R have the same combination of values for all attributes in SK. Every relation has at least one superkey: the set of all its attributes.

For $R = \{A, B, C, D\}$ with $SK = \{A, C\}$:

$t1[A,C] \neq t2[A,C]$ for any two distinct tuples $t1, t2$

Key (Candidate Key)

A key K is a minimal superkey — removing any attribute from K destroys the uniqueness property. A relation may have multiple candidate keys; one is arbitrarily chosen as the primary key.

Example (CAR relation): Key1 = {LicenseNumber}, Key2 = {EngineSerialNumber}. Both are candidate keys. {LicenseNumber, Make} is a superkey but NOT a key.

Primary Key

The chosen candidate key. Primary key attributes are conventionally underlined in schema diagrams.

2.3 Entity Integrity

No primary key attribute can be NULL. This ensures every tuple is uniquely identifiable.

2.4 Referential Integrity & Foreign Keys

A foreign key FK in $R1$ references relation $R2$ if:

- FK attributes have the same domains as $R2$'s primary key PK
- For every tuple $t1$ in $R1$: either $t1[FK] = t2[PK]$ for some $t2$ in $R2$, or $t1[FK] = NULL$

Example: The DNO attribute in EMPLOYEE is a foreign key referencing DNUMBER in DEPARTMENT. SUPERSSN in EMPLOYEE references SSN in EMPLOYEE (self-referencing).

2.5 Other Constraints

- Semantic integrity: e.g., employee salary must not exceed supervisor's salary
- Functional dependency: $X \rightarrow Y$ means value of X determines value of Y
- Transition constraints: e.g., salary can only increase

3. Update Operations & Constraint Violations

3.1 The INSERT Operation

Inserts a new tuple. Can violate domain, key, entity integrity, or referential integrity constraints.

Example Violations:

- Null in primary key field $\rightarrow$ entity integrity violation
- Duplicate SSN value $\rightarrow$ key constraint violation
- Non-existent department number $\rightarrow$ referential integrity violation

DBMS response: reject the insertion, or attempt to correct the cause.

3.2 The DELETE Operation

Removes tuples. Can only violate referential integrity if a deleted tuple is referenced by other tuples via a foreign key.

DBMS Options When Referential Integrity is Violated:

- Reject the deletion
- Cascade deletion to all referencing tuples
- Set the referencing foreign key values to NULL or DEFAULT

3.3 The UPDATE (MODIFY) Operation

Modifies attribute values. Updating a non-key, non-FK attribute rarely causes issues. Updating a primary key is like a delete + insert. Updating a FK must ensure the new value references an existing tuple in the referenced relation.

4. Relational Algebra Operations

Relational algebra is the formal language for manipulating relations. Operations are divided into two groups:

- Set operations from mathematics: UNION, INTERSECTION, SET DIFFERENCE, CARTESIAN PRODUCT
- Relational-specific operations: SELECT, PROJECT, JOIN, DIVISION

4.1 SELECT (σ) — Unary

Selects a horizontal subset of tuples satisfying a condition. Like a WHERE clause.

$\sigma_{\langle \text{condition} \rangle}(R)$

Examples:

```
 $\sigma_{DNO=4}(EMPLOYEE)$            -- employees in dept 4
 $\sigma_{SALARY>30000}(EMPLOYEE)$    -- employees earning > 30000
 $\sigma_{(DNO=4 \text{ AND } SALARY>25000) \text{ OR } (DNO=5 \text{ AND } SALARY>30000)}(EMPLOYEE)$ 
```

Properties:

- Result has same attributes as R
- Number of result tuples $\leq$ number of tuples in R
- SELECT is commutative: $\sigma_1(\sigma_2(R)) = \sigma_2(\sigma_1(R))$
- Cascade of SELECTs can be combined: $\sigma_1(\sigma_2(R)) = \sigma_{c1 \text{ AND } c2}(R)$

4.2 PROJECT (π) — Unary

Selects a vertical subset — certain columns — and discards others. Duplicate tuples are eliminated.

$\pi_{\langle \text{attribute list} \rangle}(R)$

Examples:

```
 $\pi_{LNAME, FNAME, SALARY}(EMPLOYEE)$    -- name and salary columns
 $\pi_{SEX, SALARY}(EMPLOYEE)$            -- sex and salary (duplicates removed)
```

Properties:

- Result has fewer or equal attributes and fewer or equal tuples
- $\pi_{list1}(\pi_{list2}(R)) = \pi_{list1}(R)$ if list2 contains all of list1

4.3 RENAME (ρ)

Renames relation or attribute names in a result.

```
TEMP  $\leftarrow$   $\sigma_{DNO=5}(EMPLOYEE)$ 
```

```
R(FIRSTNAME, LASTNAME, SALARY) ← πFNAME, LNAME, SALARY(TEMP)
```

4.4 SET OPERATIONS — Union, Intersection, Difference

Require type-compatible (union-compatible) relations: same degree n , and $\text{dom}(A_i) = \text{dom}(B_i)$ for all i .

UNION ($R \cup S$)

All tuples in R or S (or both). Duplicates eliminated.

INTERSECTION ($R \cap S$)

Tuples appearing in both R and S .

SET DIFFERENCE ($R - S$)

Tuples in R but NOT in S . Not commutative: $R - S \neq S - R$.

UNION and INTERSECTION are both commutative and associative.

4.5 CARTESIAN PRODUCT ($R \times S$)

Combines every tuple from R with every tuple from S combinatorially. Result has $n+m$ attributes. If $|R| = nR$ and $|S| = nS$, then $|R \times S| = nR \times nS$. Relations need NOT be type-compatible.

```
EMP_DEPENDENTS ← EMPNAMES × DEPENDENT

-- then SELECT to get matching tuples

ACTUAL_DEPENDENTS ← σSSN=ESSN(EMP_DEPENDENTS)
```

4.6 JOIN ($\bowtie$) — Binary

Combines related tuples from two relations based on a join condition. Equivalent to a CARTESIAN PRODUCT followed by a SELECT. Very important in multi-relation databases.

```
R ⋈<join condition> S
```

Types of JOIN:

THETA JOIN

General join where each condition is of the form $A_i \theta B_j$ ($\theta \in \{=, <, >, \leq, \geq, \neq\}$).

EQUIJOIN

JOIN where the only comparison operator is $=$. The join attribute appears twice in the result.

NATURAL JOIN ($R * S$)

Like EQUIJOIN, but duplicate join column is removed. Requires matching attribute names in both relations.

Example:

```
DEPT_MGR ← DEPARTMENT ⋈MGRSSN=SSN EMPLOYEE

-- Retrieves manager name for each department
```

4.7 DIVISION ($R \div S$)

Used for queries with 'for all' semantics. If $R(Z)$ and $S(X)$ where $X \subseteq Z$, then $T(Y) = R \div S$ ($Y = Z - X$) contains tuples t such that for every tuple ts in S , the combined tuple $\langle t, ts \rangle$ appears in R .

Example — Find employees who work on ALL projects:

```
T1(SSN, PNO) ←  $\pi_{\text{SSN}, \text{PNO}}$ (WORKS_ON)
T2 ←  $\pi_{\text{PNO}}$ (PROJECT)
T3 ← T1  $\div$  T2
RESULT ←  $\pi_{\text{FNAME}, \text{LNAME}}$ (T3 * EMPLOYEE)
```

4.8 AGGREGATE FUNCTIONS ($\mathcal{F}$)

Applied to collections of numeric values. Available functions: SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT.

```
 $\mathcal{F}$ COUNT SSN, AVERAGE SALARY(EMPLOYEE)

-- Returns count of employees and average salary
```

Grouping:

```
DNO  $\mathcal{F}$ COUNT SSN, AVERAGE SALARY(EMPLOYEE)

-- Groups by department number, counts and averages per department
```

4.9 Complete Set

The set $\{\sigma, \pi, \rho, -, \times\}$ is complete — all other relational algebra operations can be derived from these five.

```
 $R \cap S \equiv (R \cup S) - ((R - S) \cup (S - R))$ 

 $R \bowtie_c S \equiv \sigma_c(R \times S)$ 
```

4.10 ER-to-Relational Mapping Algorithm

A systematic 7-step process to convert an ER schema into a relational schema:

Step 1: Regular Entity Types

For each regular (strong) entity E , create a relation R with all simple attributes. Choose one key attribute as primary key.

Step 2: Weak Entity Types

For each weak entity W with owner E , create relation R . Include all simple attributes of W plus the primary key of E 's relation as foreign key. Primary key = owner's PK + partial key of W .

Step 3: Binary 1:1 Relationships

Choose one participating relation S (preferably with total participation). Add the primary key of the other relation T as a foreign key in S . Add relationship attributes to S .

Step 4: Binary 1:N Relationships

Identify the N-side relation S. Add the primary key of the 1-side relation T as a foreign key in S. Add relationship attributes to S.

Step 5: Binary M:N Relationships

Create a new relation S for the relationship. Include primary keys of both participating relations as foreign keys (together they form the primary key). Add relationship attributes.

Step 6: Multivalued Attributes

Create a new relation R for each multivalued attribute A. Include A as an attribute plus the primary key K of its owner as a foreign key. Primary key of R = {A, K}.

Step 7: N-ary Relationships ($n > 2$)

Create a new relation S. Include foreign keys for all participating entity types' primary keys. Add any simple relationship attributes.

5. Introduction to Database Design

Database design evaluates relational schemas for quality at two levels:

- Logical (conceptual) level: how users interpret schemas and attribute meanings
- Implementation (storage) level: how tuples in base relations are stored and updated

Two design methodologies exist: bottom-up (design by synthesis) and top-down (design by analysis).

6. Four Informal Design Guidelines

Guideline 1 — Clear Semantics

Design schemas so that it is easy to explain their meaning. Do not mix attributes from multiple entity types and relationship types into a single relation.

Poor design example: Combining EMPLOYEE and DEPARTMENT into EMP_DEPT, or merging EMPLOYEE, PROJECT, and WORKS_ON into EMP_PROJ, violates this guideline.

Guideline 2 — Avoid Update Anomalies

Do not design schemas that introduce insertion, deletion, or modification anomalies.

Types of Update Anomalies:

Insertion Anomaly (Consistency)

To insert a new employee in EMP_DEPT, you must correctly repeat all department attribute values, risking inconsistency.

Insertion Anomaly (Null Values)

Cannot insert a department with no employees yet in EMP_DEPT, because SSN (PK) cannot be null.

Deletion Anomaly

Deleting the last employee of a department from EMP_DEPT also destroys the department information.

Modification Anomaly

Changing the manager of department 5 requires updating ALL employee tuples of that department, risking inconsistency.

Guideline 3 — Minimize NULLs

Avoid placing attributes in a base relation whose values may frequently be NULL. Null values waste storage, create problems in JOIN operations, and complicate aggregate functions like COUNT and SUM.

Guideline 4 — Avoid Spurious Tuples

Design schemas so that JOINS are performed on primary key / foreign key pairs. Joining on non-key matching attributes can generate spurious (invalid) tuples that do not represent real data.

Example: Splitting EMP_PROJ into EMP_PROJ1 and EMP_LOCS using PLOCATION as a join attribute (which is neither PK nor FK in either table) produces spurious tuples when re-joined.

7. Functional Dependencies (FDs)

A functional dependency is a constraint between two attribute sets in the database.

7.1 Formal Definition

Let R be a relation schema, X and Y be attribute subsets of R. A functional dependency $X \rightarrow Y$ specifies that for any two tuples t1, t2 in r(R):

if t1[X] = t2[X], **then** t1[Y] = t2[Y]

In other words: the values of X uniquely determine the values of Y. We say Y is functionally dependent on X.

Key property: If X is a candidate key of R, then $X \rightarrow Y$ for any subset Y of R.

7.2 Example FDs

EMP_PROJ(SSN, PNUMBER, HOURS, ENAME, PNAME, PLOCATION)

FD1: {SSN, PNUMBER} $\rightarrow$ HOURS

FD2: SSN $\rightarrow$ ENAME

FD3: PNUMBER $\rightarrow$ {PNAME, PLOCATION}

7.3 Closure of FDs (F+)

The closure of a set F, denoted F+, is the set of all FDs that include F plus all FDs that can be inferred from F. For example, from SSN $\rightarrow$ DNUMBER and DNUMBER $\rightarrow$ DNAME, we can infer SSN $\rightarrow$ DNAME.

7.4 Armstrong's Inference Rules (IR1–IR3)

IR1: Reflexive Rule

If $Y \subseteq X$, then $X \rightarrow Y$ (a set always determines its own subsets).

IR2: Augmentation Rule

$\{X \rightarrow Y\}$ implies $XZ \rightarrow YZ$ (add same attributes to both sides).

IR3: Transitive Rule

$\{X \rightarrow Y, Y \rightarrow Z\}$ implies $X \rightarrow Z$.

Additional derived rules:

- IR4 (Decomposition): $\{X \rightarrow YZ\}$ implies $X \rightarrow Y$
- IR5 (Union): $\{X \rightarrow Y, X \rightarrow Z\}$ implies $X \rightarrow YZ$
- IR6 (Pseudotransitive): $\{X \rightarrow Y, WY \rightarrow Z\}$ implies $WX \rightarrow Z$

IR1–IR3 are called Armstrong's rules and are both sound and complete.

7.5 Computing X^+ (Closure of Attribute Set X under F)

Algorithm: Determining X^+ under F

$X^+ = X$

repeat

$\text{old}X^+ = X^+$

 for each FD $Y \rightarrow Z$ in F :

 if $Y \subseteq X^+$ then $X^+ = X^+ \cup Z$

until $(X^+ = \text{old}X^+)$

Example:

$F = \{SSN \rightarrow ENAME, PNUMBER \rightarrow \{PNAME, PLOCATION\}, \{SSN, PNUMBER\} \rightarrow HOURS\}$

$\{SSN\}^+ = \{SSN, ENAME\}$

$\{PNUMBER\}^+ = \{PNUMBER, PNAME, PLOCATION\}$

$\{SSN, PNUMBER\}^+ = \{SSN, PNUMBER, HOURS, ENAME, PNAME, PLOCATION\}$

7.6 Equivalence of FD Sets

F covers E if every FD in E is in F^+ . F and E are equivalent if $E^+ = F^+$ (each covers the other).

Testing Equivalence — Steps:

- For each FD $X \rightarrow Y$ in E , compute X^+ using F and check $Y \subseteq X^+$ (does F cover E ?)
- Repeat: for each FD $A \rightarrow B$ in F , compute A^+ using E and check $B \subseteq A^+$ (does E cover F ?)
- If both hold, F and E are equivalent.

7.7 Minimal Cover of F

A set F is minimal if: (1) every dependency has a single attribute on the right-hand side, (2) no left-hand side attribute is redundant, and (3) no entire dependency is redundant.

Algorithm — Finding Minimal Cover:

1. Replace $X \rightarrow \{A_1, A_2, \dots, A_n\}$ with n separate FDs: $X \rightarrow A_1, X \rightarrow A_2, \dots$

2. For each $X \rightarrow A$ in F :

For each attribute B in X:

if $(F - \{X \rightarrow A\}) \cup \{(X - \{B\}) \rightarrow A\}$ is equivalent to F:

replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$

3. For each remaining $X \rightarrow A$:

if $(F - \{X \rightarrow A\})$ is equivalent to F:

remove $X \rightarrow A$ from F

8. Normalization & Normal Forms

Normalization analyzes relation schemas based on their FDs and primary keys to minimize redundancy and anomalies. It decomposes schemas into smaller, better-designed schemas.

Two key properties of good decompositions:

Lossless (Nonadditive) Join Property

No spurious tuples are generated when decomposed relations are re-joined.

Dependency Preservation Property

Each functional dependency is represented in some individual relation after decomposition.

8.1 Key Definitions

Prime Attribute

An attribute that is a member of some candidate key of R.

Non-prime Attribute

An attribute that is NOT a member of any candidate key.

Example: In WORKS_ON(SSN, PNUMBER, HOURS), SSN and PNUMBER are prime; HOURS is non-prime.

8.2 First Normal Form (1NF)

A relation is in 1NF if the domain of every attribute contains only atomic (simple, indivisible) values. 1NF disallows multivalued attributes, composite attributes, and nested relations.

Techniques to Achieve 1NF for Multivalued Attributes:

- Option 1 (Best): Remove the multivalued attribute to a separate relation with the original primary key as foreign key.
- Option 2: Expand the key — create a separate tuple for each value, making the multivalued attribute part of the primary key (causes redundancy).

- Option 3: Replace the multivalued attribute with N separate atomic attributes if maximum N values are known (causes null issues and spurious tuples).

8.3 Full vs. Partial Functional Dependency

Full Functional Dependency

$X \rightarrow Y$ is full if removing any attribute A from X causes the dependency to no longer hold.

Partial Functional Dependency

$X \rightarrow Y$ is partial if some attribute A can be removed from X and the dependency still holds.

Example (EMP_PROJ):

```
{SSN, PNUMBER} → HOURS      -- FULL dependency
{SSN, PNUMBER} → ENAME       -- PARTIAL (because SSN → ENAME holds)
```

8.4 Second Normal Form (2NF)

A relation schema R is in 2NF if every non-prime attribute A in R is FULLY functionally dependent on every candidate key of R. If any non-prime attribute has a partial dependency on a key, the schema is NOT in 2NF.

How to Achieve 2NF:

Decompose the relation by associating each group of attributes with the partial key they fully depend on.

EMP_PROJ is NOT in 2NF due to FD2 (SSN → ENAME) and FD3 (PNUMBER → {PNAME, PLOCATION}).

Decompose into:

```
EP1(SSN, PNUMBER, HOURS)      -- full dependency on {SSN, PNUMBER}
EP2(SSN, ENAME)               -- depends only on SSN
EP3(PNUMBER, PNAME, PLOCATION) -- depends only on PNUMBER
```

8.5 Transitive Dependency

Transitive Dependency

$X \rightarrow Y$ is transitive in R if there exists Z (not a candidate key, not a subset of any key) such that $X \rightarrow Z$ and $Z \rightarrow Y$ both hold.

Example (EMP_DEPT):

SSN → DMGRSSN is TRANSITIVE through DNUMBER:

```
SSN → DNUMBER and DNUMBER → DMGRSSN
```

DNUMBER is not a key, making this a transitive dependency.

8.6 Third Normal Form (3NF)

A relation schema R is in 3NF if it satisfies 2NF AND no non-prime attribute of R is transitively dependent on the primary key.

General definition: R is in 3NF if, whenever a nontrivial FD $X \rightarrow A$ holds in R, either:

- X is a superkey of R, OR
- A is a prime attribute of R

Achieving 3NF:

EMP_DEPT has transitive dependencies:

$SSN \rightarrow DNUMBER \rightarrow DMGRSSN$

$SSN \rightarrow DNUMBER \rightarrow DNAME$

Decompose into:

ED1(SSN, ENAME, BDATE, ADDRESS, SEX, SALARY, SUPERSSN, DNO)

ED2(DNUMBER, DNAME, MGRSSN, MGRSTARTDATE)

NATURAL JOIN on DNO=DNUMBER recovers EMP_DEPT without spurious tuples.

8.7 Boyce-Codd Normal Form (BCNF)

BCNF is a stronger version of 3NF. A relation schema R is in BCNF if, whenever a nontrivial FD $X \rightarrow A$ holds in R, X must be a superkey of R. The difference from 3NF: BCNF does NOT allow A to be a prime attribute.

A relation may be in 3NF but not BCNF when it has multiple overlapping candidate keys.

When BCNF Decomposition May Lose Dependency Preservation:

TEACH(STUDENT, COURSE, INSTRUCTOR)

FD1: {STUDENT, COURSE} $\rightarrow$ INSTRUCTOR (candidate key)

FD2: INSTRUCTOR $\rightarrow$ COURSE

This is in 3NF (FD2: COURSE is prime) but NOT in BCNF (INSTRUCTOR is not superkey).

Desirable BCNF decomposition:

{INSTRUCTOR, STUDENT} and {INSTRUCTOR, COURSE}

Note: FD1 is LOST in this decomposition.

Summary of Normal Forms:

1NF $\rightarrow$ 2NF (remove partial dependencies) $\rightarrow$ 3NF (remove transitive dependencies) $\rightarrow$ BCNF (every determinant must be a superkey).

Structured Query Language (SQL-99)

9. Introduction to SQL

SQL (Structured Query Language) is a high-level declarative language for relational databases. It is both a DDL (Data Definition Language) and a DML (Data Manipulation Language).

SQL can:

- Define data — CREATE TABLE, CREATE SCHEMA, CREATE VIEW
- Query data — SELECT-FROM-WHERE
- Manipulate data — INSERT, DELETE, UPDATE
- Define views, integrity constraints, triggers, and assertions
- Specify security, authorization, and transaction controls
- Be embedded in programming languages (C, C++, Java, Visual Basic)

In SQL terminology: relation → table; tuple → row; attribute → column.

10. SQL Data Definition

10.1 Schema

```
CREATE SCHEMA COMPANY AUTHORIZATION JSMITH;
```

A schema contains tables, views, domains, constraints, and other elements. Catalogs are named collections of schemas and always contain INFORMATION_SCHEMA.

10.2 CREATE TABLE

Specifies a new table with its attributes and constraints. Attributes must be given a name, data type, and optional constraints.

```
CREATE TABLE EMPLOYEE (  
  
    SSN          CHAR(9)          NOT NULL,  
  
    FNAME        VARCHAR(15)      NOT NULL,  
  
    LNAME        VARCHAR(15)      NOT NULL,  
  
    BDATE        DATE ,  
  
    ADDRESS      VARCHAR(30) ,  
  
    SEX          CHAR ,  
  
    SALARY        DECIMAL(10,2) ,  
  
    SUPERSSN     CHAR(9) ,
```



```

DNO          INT          NOT NULL DEFAULT 1,

CONSTRAINT EMPFK PRIMARY KEY(SSN),

CONSTRAINT EMPSUPERFK FOREIGN KEY(SUPERSSN) REFERENCES EMPLOYEE(SSN)

ON DELETE SET NULL ON UPDATE CASCADE,

CONSTRAINT EMPDEPTFK FOREIGN KEY(DNO) REFERENCES DEPARTMENT(DNUMBER)

ON DELETE SET DEFAULT ON UPDATE CASCADE

);

```

10.3 SQL Data Types

Numeric:

- INTEGER or INT, SMALLINT
- FLOAT or REAL, DOUBLE PRECISION
- DECIMAL(i, j) — i total digits, j after decimal

Character:

- CHAR(n) — fixed-length string
- VARCHAR(n) — variable-length string
- Literals in single quotes: 'SMITH' (case-sensitive)

Bit String:

- BIT(n) — fixed-length; BIT VARYING(n) — variable-length
- Literals prefixed with B: B'10101'

Date/Time:

```

DATE          -- YYYY-MM-DD (10 chars)

TIME          -- HH:MM:SS (8+ chars)

TIMESTAMP     -- DATE + TIME + optional decimals

INTERVAL      -- relative value (YEAR/MONTH or DAY/TIME)

```

Boolean:

Values: TRUE, FALSE, or UNKNOWN (three-valued logic).

10.4 CREATE DOMAIN

```

CREATE DOMAIN SSN_TYPE AS CHAR(9);

-- Use: SSN SSN_TYPE NOT NULL  (instead of SSN CHAR(9) NOT NULL)

```

10.5 Constraints in SQL

Attribute-Level:

```

NOT NULL          -- value cannot be null

```

```
DEFAULT 1                -- default value

CHECK (DNUMBER > 0 AND DNUMBER < 21)  -- domain restriction
```

Table-Level:

```
PRIMARY KEY (DNUMBER, DLOCATION)      -- composite primary key

UNIQUE (DNAME)                      -- alternate key

FOREIGN KEY (DNO) REFERENCES DEPARTMENT(DNUMBER)
```

Referential Action Options:

```
ON DELETE SET NULL

ON DELETE SET DEFAULT

ON DELETE CASCADE

ON UPDATE CASCADE
```

Tuple-Level CHECK:

```
CHECK (DEPT_CREATE_DATE < MGRSTARTDATE)  -- at end of CREATE TABLE
```

Named Constraints:

```
CONSTRAINT DEPTPK PRIMARY KEY(DNUMBER)  -- name allows drop/modify later
```

10.6 Schema Change — DROP & ALTER

```
-- DROP

DROP SCHEMA COMPANY CASCADE;          -- removes schema + all contents

DROP TABLE DEPENDENT CASCADE;        -- removes table + constraints/views referencing it

DROP TABLE EMPLOYEE RESTRICT;        -- fails if referenced elsewhere


-- ALTER TABLE

ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12);

ALTER TABLE EMPLOYEE DROP ADDRESS CASCADE;

ALTER TABLE DEPARTMENT ALTER MGRSSN DROP DEFAULT;

ALTER TABLE DEPARTMENT ALTER MGRSSN SET DEFAULT '333445555';

ALTER TABLE EMPLOYEE DROP CONSTRAINT EMPSUPERFK CASCADE;
```

11. Basic Queries in SQL

11.1 SELECT-FROM-WHERE

```
SELECT <attribute list>

FROM <table list>

WHERE <condition>
```

An SQL table is NOT a set — duplicate rows can exist. Logical comparison operators: =, >, <, >=, <=, <> (not equal).

11.2 Example Queries

Query 0 — Single table:

```
SELECT BDATE, ADDRESS FROM EMPLOYEE

WHERE FNAME='John' AND MINIT='B' AND LNAME='Smith';
```

Query 1 — JOIN (two tables):

```
SELECT FNAME, LNAME, ADDRESS

FROM EMPLOYEE, DEPARTMENT

WHERE DNUMBER=DNO AND DNAME='Research';

-- DNUMBER=DNO is the join condition
-- DNAME='Research' is the selection condition
```

Query 2 — Multi-table JOIN:

```
SELECT PNUMBER, DNUM, LNAME, BDATE, ADDRESS

FROM PROJECT, DEPARTMENT, EMPLOYEE

WHERE DNUM=DNUMBER AND MGRSSN=SSN AND PLOCATION='Stafford';
```

11.3 Disambiguation & Aliasing

Qualifying Attribute Names:

```
SELECT FNAME, EMPLOYEE.NAME, ADDRESS

FROM EMPLOYEE, DEPARTMENT

WHERE DEPARTMENT.NAME='Research' AND EMPLOYEE.DNUMBER=DEPARTMENT.DNUMBER;
```

Table Aliases (Tuple Variables):

```
-- Self-join: employee and their supervisor

SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME

FROM EMPLOYEE AS E, EMPLOYEE AS S

WHERE E.SUPERSSN=S.SSN;
```

11.4 Special Clauses

Wildcard (*):

```
SELECT * FROM EMPLOYEE WHERE DNO=5;    -- all columns
```

DISTINCT:

```
SELECT DISTINCT SALARY FROM EMPLOYEE; -- removes duplicate salary values
```

ORDER BY:

```
SELECT DNAME, LNAME, FNAME, PNAME
FROM DEPARTMENT, EMPLOYEE, WORKS_ON, PROJECT
WHERE DNUMBER=DNO AND SSN=ESSN AND PNO=PNUMBER
ORDER BY DNAME, LNAME, FNAME; -- ASC is default; use DESC for descending
```

BETWEEN:

```
WHERE SALARY BETWEEN 30000 AND 40000 -- equivalent to >= 30000 AND <= 40000
```

Arithmetic Operators:

```
SELECT FNAME, LNAME, 1.1 * SALARY AS INCREASED_SAL
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE SSN=ESSN AND PNO=PNUMBER AND PNAME='ProductX';
```

LIKE (Pattern Matching):

```
-- % matches any sequence; _ matches single character
WHERE ADDRESS LIKE '%Houston%' -- contains 'Houston'
WHERE BDATE LIKE '195_____' -- born in 1950s (YYYY-MM-DD)
```

11.5 Set Operations

```
(SELECT PNUMBER FROM PROJECT WHERE ...) UNION
(SELECT PNUMBER FROM PROJECT, DEPARTMENT WHERE ...)

-- Also: EXCEPT (set difference), INTERSECT (intersection)
-- UNION ALL, EXCEPT ALL, INTERSECT ALL keep duplicates (multiset operations)
```

12. Complex Queries

12.1 NULL Values & Three-Valued Logic

SQL uses three truth values: TRUE, FALSE, UNKNOWN. Comparisons with NULL yield UNKNOWN. Use IS NULL or IS NOT NULL (not = NULL).

```
SELECT FNAME, LNAME FROM EMPLOYEE
WHERE SUPERSSN IS NULL; -- employees with no supervisor
```

12.2 Nested Queries (Subqueries)

A nested query is a complete SELECT-FROM-WHERE block inside the WHERE clause of an outer query. The inner query runs first.

Using IN operator:

```
SELECT DISTINCT PNUMBER FROM PROJECT
WHERE PNUMBER IN
    (SELECT PNUMBER FROM PROJECT, DEPARTMENT, EMPLOYEE
     WHERE DNUM=DNUMBER AND MGRSSN=SSN AND LNAME='Smith')
OR PNUMBER IN
    (SELECT PNO FROM WORKS_ON, EMPLOYEE
     WHERE ESSN=SSN AND LNAME='Smith');
```

Using ANY / SOME / ALL:

```
-- Employees earning more than ALL dept-5 employees:
SELECT LNAME, FNAME FROM EMPLOYEE
WHERE SALARY > ALL (SELECT SALARY FROM EMPLOYEE WHERE DNO=5);
```

12.3 Correlated Nested Queries

When the nested query references an attribute from the outer query, they are correlated. The inner query is evaluated once for each tuple in the outer query.

```
-- Employees with dependents of the same name and sex:
SELECT E.FNAME, E.LNAME FROM EMPLOYEE AS E
WHERE E.SSN IN
    (SELECT ESSN FROM DEPENDENT
     WHERE E.FNAME=DEPENDENT_NAME AND E.SEX=SEX);
```

12.4 EXISTS and NOT EXISTS

```
-- Using EXISTS (returns TRUE if subquery result is non-empty):
SELECT E.FNAME, E.LNAME FROM EMPLOYEE AS E
WHERE EXISTS (SELECT * FROM DEPENDENT
              WHERE E.SSN=ESSN AND E.SEX=SEX
              AND E.FNAME=DEPENDENT_NAME);

-- Using NOT EXISTS (employees with no dependents):
SELECT FNAME, LNAME FROM EMPLOYEE
WHERE NOT EXISTS (SELECT * FROM DEPENDENT WHERE SSN=ESSN);
```

```
-- Managers with at least one dependent:

SELECT FNAME, LNAME FROM EMPLOYEE

WHERE EXISTS (SELECT * FROM DEPENDENT WHERE SSN=ESSN)

      AND EXISTS (SELECT * FROM DEPARTMENT WHERE SSN=MGRSSN);
```

12.5 Renaming Output Attributes

```
SELECT E.LNAME AS EMPLOYEE_NAME, S.LNAME AS SUPERVISOR_NAME

FROM EMPLOYEE AS E, EMPLOYEE AS S

WHERE E.SUPERSSN=S.SSN;
```

12.6 Joined Tables Syntax

```
-- JOIN keyword in FROM clause:

SELECT FNAME, LNAME, ADDRESS

FROM (EMPLOYEE JOIN DEPARTMENT ON DNO=DNUMBER)

WHERE DNAME='Research';
```

```
-- NATURAL JOIN:

SELECT FNAME, LNAME, ADDRESS

FROM (EMPLOYEE NATURAL JOIN

      (DEPARTMENT AS DEPT(DNAME, DNO, MSSN, MSDATE)))

WHERE DNAME='Research';
```

```
-- Multiple JOINS:

SELECT PNUMBER, DNUM, LNAME, ADDRESS, BDATE

FROM ((PROJECT JOIN DEPARTMENT ON DNUM=DNUMBER)

      JOIN EMPLOYEE ON MGRSSN=SSN)

WHERE PLOCATION='Stafford';
```

13. Aggregate Functions

COUNT, SUM, MAX, MIN, AVG — applied in SELECT clause or HAVING clause.

-- Basic aggregates:

```
SELECT SUM(SALARY), MAX(SALARY), MIN(SALARY), AVG(SALARY) FROM EMPLOYEE;
```

-- Count all tuples:

```
SELECT COUNT(*) FROM EMPLOYEE;
```

-- Count distinct values (NULLs excluded):

```
SELECT COUNT(DISTINCT SALARY) FROM EMPLOYEE;
```

-- Employees with 2+ dependents:

```
SELECT LNAME, FNAME FROM EMPLOYEE
```

```
WHERE (SELECT COUNT(*) FROM DEPENDENT WHERE SSN=ESSN) >= 2;
```

13.1 GROUP BY and HAVING

-- Group by department, count and average salary:

```
SELECT DNO, COUNT(*), AVG(SALARY)
```

```
FROM EMPLOYEE
```

```
GROUP BY DNO;
```

-- Projects with more than 2 employees:

```
SELECT PNUMBER, PNAME, COUNT(*)
```

```
FROM PROJECT, WORKS_ON
```

```
WHERE PNUMBER=PNO
```

```
GROUP BY PNUMBER, PNAME
```

```
HAVING COUNT(*) > 2;
```

-- Complex: dept 5 employees on each project:

```
SELECT PNUMBER, PNAME, COUNT(*)
```

```
FROM PROJECT, WORKS_ON, EMPLOYEE
```

```
WHERE PNUMBER=PNO AND SSN=ESSN AND DNO=5
```

```
GROUP BY PNUMBER, PNAME;
```

Rule: Grouping attributes must appear in the SELECT clause. HAVING filters groups (applied after GROUP BY), while WHERE filters rows (applied before GROUP BY).

14. INSERT, DELETE, UPDATE

14.1 INSERT

```
-- Standard insert (all attributes in order):
```

```
INSERT INTO EMPLOYEE
```

```
VALUES ('Richard','K','Marini','653298653','1962-12-30',  
        '98 Oak Forest,Katy,TX','M',37000,'987654321',4);
```

```
-- Explicit attribute list (NULLs/defaults for omitted attributes):
```

```
INSERT INTO EMPLOYEE (FNAME, LNAME, DNO, SSN)
```

```
VALUES ('Richard','Marini',4,'653298653');
```

```
-- Insert result of a query:
```

```
INSERT INTO DEPTS_INFO (DEPT_NAME, NO_OF_EMPS, TOTAL_SAL)
```

```
SELECT DNAME, COUNT(*), SUM(SALARY)
```

```
FROM DEPARTMENT, EMPLOYEE
```

```
WHERE DNUMBER=DNO
```

```
GROUP BY DNAME;
```

14.2 DELETE

```
DELETE FROM EMPLOYEE WHERE LNAME='Brown';
```

```
DELETE FROM EMPLOYEE WHERE SSN='123456789';
```

```
-- Using a subquery:
```

```
DELETE FROM EMPLOYEE
```

```
WHERE DNO IN (SELECT DNUMBER FROM DEPARTMENT WHERE DNAME='Research');
```



```
-- Delete all rows:

DELETE FROM EMPLOYEE;
```

14.3 UPDATE

```
-- Update specific tuple:

UPDATE PROJECT SET PLOCATION='Bellaire', DNUM=5 WHERE PNUMBER=10;

-- Update multiple tuples:

UPDATE EMPLOYEE SET SALARY = SALARY * 1.1

WHERE DNO IN (SELECT DNUMBER FROM DEPARTMENT WHERE DNAME='Research');
```

15. Views (Virtual Tables)

A view is a named derived table based on other tables or views. It does not physically store data but is always logically up to date.

15.1 Creating Views

```
-- Create a view joining EMPLOYEE, WORKS_ON, PROJECT:

CREATE VIEW WORKS_ON_INFO AS

    SELECT FNAME, LNAME, PNAME, HOURS

    FROM EMPLOYEE, WORKS_ON, PROJECT

    WHERE SSN=ESSN AND PNO=PNUMBER;

-- Query the view like a table:

SELECT FNAME, LNAME

FROM WORKS_ON_INFO

WHERE PNAME='ProductX';

-- Remove a view:

DROP VIEW WORKS_ON_INFO;
```

Advantages: simplifies complex queries, provides a security mechanism (restrict user access to certain columns/rows), and can have different attribute names.

16. Assertions and Triggers

16.1 Assertions

CREATE ASSERTION specifies additional constraints beyond built-in SQL constraints. The condition must always be TRUE.

```
-- Salary of employee must not exceed manager's salary:
```

```
CREATE ASSERTION SALARY_CONSTRAINT
```

```
CHECK (NOT EXISTS
```

```
  (SELECT * FROM EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
```

```
   WHERE E.Salary > M.Salary
```

```
        AND E.Dno=D.Dnumber
```

```
        AND D.Mgr_ssn=M.Ssn));
```

```
-- Sum of loans per customer must not exceed 100,000:
```

```
CREATE ASSERTION sumofloans
```

```
CHECK (100000 >= ALL (
```

```
  SELECT customer_name, sum(amount)
```

```
  FROM borrower b, loan l
```

```
  WHERE b.loan_number=l.loan_number
```

```
  GROUP BY customer_name));
```

16.2 Triggers

A trigger executes automatically when a specified event occurs. It has three components: Event, Condition (optional), and Action.

```
-- General form:
```

```
CREATE TRIGGER <name>
```

```
BEFORE | AFTER <event: INSERT | UPDATE | DELETE>
```

```
[FOR EACH ROW | FOR EACH STATEMENT]
```

```
[WHEN (<condition>)]
```

```
<action>;
```

```
-- Example: Salary violation trigger:
```

```
CREATE TRIGGER SALARY_VIOLATION  
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN  
ON EMPLOYEE  
FOR EACH ROW  
WHEN (NEW.SALARY > (SELECT SALARY FROM EMPLOYEE  
                     WHERE SSN=NEW.SUPERVISOR_SSN))  
INFORM_SUPERVISOR(NEW.Supervisor_ssn, NEW.Ssn);
```

16.3 Assertions vs. Triggers

- Assertions: do not modify data; only check conditions; not linked to specific tables or events
- Triggers: can check conditions AND modify data; linked to specific tables and events
- All assertions can be implemented as triggers, but not vice versa

17. Normal Forms Summary

Normal Form	Requirement	How to Achieve
1NF	All attribute values must be atomic (no multivalued or composite attributes)	Move multivalued attributes to a separate relation
2NF	In 1NF + no non-prime attribute has a partial dependency on any candidate key	Remove partial dependencies; create new relations for each partial key
3NF	In 2NF + no non-prime attribute has a transitive dependency on any candidate key	Remove transitive dependencies; create new relations
BCNF	Whenever $X \rightarrow A$ (nontrivial) holds, X must be a superkey of R	Decompose; may lose some FD preservation

18. Relational Algebra Quick Reference

Operation	Symbol / Syntax	Description
SELECT	$\sigma_{\langle \text{cond} \rangle}(R)$	Filters rows; same schema as R; number of tuples $\leq R $
PROJECT	$\pi_{\langle \text{attrs} \rangle}(R)$	Selects columns; removes duplicates; degree = attrs
RENAME	$\rho_{\langle \text{new} \rangle}(R)$	Renames relation or attributes
UNION	$R \cup S$	All tuples in R or S; requires type compatibility; removes duplicates
INTERSECTION	$R \cap S$	Tuples in both R and S; type-compatible
DIFFERENCE	$R - S$	Tuples in R but not S; not commutative
CARTESIAN	$R \times S$	All combinations; result has n+m attributes; $ \text{result} = R \times S $
JOIN	$R \bowtie_c S$	SELECT(c) applied to CARTESIAN PRODUCT; only matching tuples
NATURAL JOIN	$R \star S$	EQUIJOIN + removes duplicate join attribute(s)
DIVISION	$R \div S$	'For all' semantics; result $Y = Z - X$ where $X = \text{attrs}(S)$, $Z = \text{attrs}(R)$

19. SQL Command Summary

Category	Commands
----------	----------

DDL	CREATE SCHEMA, CREATE TABLE, CREATE DOMAIN, CREATE VIEW, CREATE ASSERTION, CREATE TRIGGER, ALTER TABLE, DROP TABLE, DROP SCHEMA
DML — Query	SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY
DML — Modify	INSERT INTO, DELETE FROM, UPDATE ... SET
Constraints	NOT NULL, DEFAULT, CHECK, PRIMARY KEY, UNIQUE, FOREIGN KEY, ON DELETE/UPDATE CASCADE/SET NULL/SET DEFAULT
Subqueries	IN, NOT IN, ANY, ALL, EXISTS, NOT EXISTS, correlated subqueries
Set Ops	UNION, EXCEPT, INTERSECT (set); UNION ALL, EXCEPT ALL, INTERSECT ALL (multiset)
Aggregates	COUNT, SUM, AVG, MAX, MIN — use with GROUP BY and HAVING
Joins	INNER JOIN ON, NATURAL JOIN, LEFT/RIGHT/FULL OUTER JOIN